PLC_1 (Line 1) — step-by-step build guide (TIA Portal V18)

A true click-by-click guide. Do it top to bottom in one sitting. We **start from your existing** `CraneSim.ap18` (it already has the shared blocks + `Line4Data`), duplicate it, and turn it into PLC_1. That way you never re-type the big data block by hand.

What you're building: PLC_1 controls **Line 1**, which has **two cranes** — **1A** (entry, has loading bays) and **1B** (downstream). Each crane needs its own data block + its own copy of the sequencer. The motion logic is identical to Line 4; only the data-block names and the I/O wiring change.

End state (what the project tree will look like when done):

```
PLC_1 [CPU 1511-1 PN]
├─ Program blocks
│   ├─ Main [OB1]              ← calls IOMap, then Crane1A_Seq, then Crane1B_Seq
│   ├─ IOMap [FC]              ← NEW: wires real %I/%Q to the DB mirror (paste
IOMap.scl)
│   ├─ TravelTo [FC]           ← reused unchanged (from CraneSim)
│   ├─ Crane1A_Seq [FB]        ← copy of CraneSequencer, points at Crane1ADataa
│   ├─ Crane1B_Seq [FB]        ← copy of CraneSequencer, points at Crane1BData
│   ├─ Crane1AData [DB]        ← copy of Line4Data, renamed
│   └─ Crane1BData [DB]        ← copy of Line4Data, renamed
└─ PLC data types
    └─ typeTask                ← reused unchanged (from CraneSim)
```

Step 1 — Duplicate CraneSim into a PLC_1 project

1. Open **TIA Portal V18**.
2. Menu **Project** → **Open...**, browse to `D:\MyGitHub\crane_automation\CraneSim\CraneSim.ap18`, click **Open**.
3. Wait for it to load, then menu **Project** → **Save as...**.
4. In the dialog: **File name** = `PLC_1_Line1`, choose a folder (e.g. `D:\MyGitHub\crane_automation`), click **Save**.
    - You are now working in the **new** project. `CraneSim` is untouched.
5. In the left **Project tree**, expand your PLC device → **Program blocks** and **PLC data types**. You should already see `typeTask`, `TravelTo`, `CraneSequencer`, `Line4Data` (and maybe `PlcData`). Those are the blocks we'll reuse and copy.

> If you'd rather build from a blank project instead of duplicating, see "Alternative: fresh project" at the bottom. Duplicating is faster because `Line4Data` (25+ rows) already exists.

Step 2 — Set this PLC's IP address

1. In the Project tree, double-click **Device configuration** (under the PLC).

2. In the graphical **Device view**, click the CPU, then the green **PROFINET interface** port.
3. In the bottom **Properties** pane → **General** → **PROFINET interface [X1]** → **Ethernet addresses**.
4. Set **IP address** = `192.168.0.111`, **Subnet mask** = `255.255.255.0`.
    - (For the PLCSIM sim you can keep the sim IP `192.168.50.1` — the real target is `.111`.)
5. Still in Properties → **OPC UA** → **Server**: confirm **Activate OPC UA server** is ticked. The endpoint shows as `opc.tcp://…:4840`. Leave security at defaults for now (that's Step 9).

Step 3 — Add the I/O modules (so %I/%Q exist)

Use the **exact modules from Panel_1** (`raza_docs/Panel_Drawings_081426 (1).xlsx`) — do **not** pick a Basic (BA) or 16-channel variant:

Panel role	Catalog folder to expand	Article number	Gives addresses
Input Card 1 (crane 1A)	**DI 32x24VDC HF**	`6ES7521-1BL00-0AB0`	`%I0.0…%I3.7`
Input Card 2 (crane 1B)	**DI 32x24VDC HF**	`6ES7521-1BL00-0AB0`	`%I4.0…%I7.7`
Output Card	**DQ 32x24VDC HF**	`6ES7522-1BL01-0AB0`	`%Q0.0…%Q3.7`

1. In **Device view**, open the **Hardware catalog** (right side) → **DI/DQ** → expand **`DI 32x24VDC HF`**.
2. Drag **`6ES7521-1BL00-0AB0`** onto the first free slot next to the CPU (this is crane 1A → `%I0`). Drag a **second** `6ES7521-1BL00-0AB0` onto the next slot (crane 1B → `%I4`).
3. Expand **`DQ 32x24VDC HF`** and drag **`6ES7522-1BL01-0AB0`** onto the next slot (`%Q0`).
4. If the catalog offers a firmware picker, match the module firmware to what you have; if unsure use the latest. Then click each module → **Properties** → **I/O addresses** and confirm the start bytes are **0 / 4 / 0** so the ranges cover the `%I`/`%Q` used in `./IOMap.scl` (shift the start address if not).

> **Why 32-channel + two DI cards:** `IOMap.scl` addresses `%I0.0…%I7.7` (64 inputs = crane 1A on `%I0–%I3`, crane 1B on `%I4–%I7`), so you need **two** 32-ch DI cards; a 16-ch card can't reach `%I7`. `HF` (High Feature) is the panel's article `…-0AB0`; the `BA` (Basic) folders are a different article.

Step 4 — Create the two crane data blocks (copy Line4Data twice)

Crane 1A first:

1. Project tree → **Program blocks** → right-click **`Line4Data`** → **Copy**.
2. Right-click **Program blocks** → **Paste**. A `Line4Data_1` appears.
3. Right-click it → **Rename** (or press **F2**) → type `Crane1AData` → Enter.

Crane 1B — repeat: 4. Copy `Line4Data` again, Paste, rename the copy to **`Crane1BData`**.

Now confirm each new DB is OPC-UA ready (do this for **both** `Crane1AData` and `Crane1BData`): 5. Double-click the DB to open it. Right-click the DB in the tree → **Properties** → **Attributes** → confirm **Optimized block access** is **ticked**. 6. In the DB table you'll see columns **Accessible from HMI/OPC UA** and **Writable from HMI/OPC UA**. Make sure these rows have **both** boxes ticked: `task_count`, `cmd_send`, and inside `tasks` the `station` and `dwell_s` fields. (The `sts_*` rows need only *Accessible*.)

- If they're already ticked (inherited from Line4Data), great — just verify.

Step 5 — Create the two sequencer FBs (copy + one find/replace)

Crane 1A:

1. Project tree → right-click `CraneSequencer` → **Copy** → right-click **Program blocks** → **Paste** → rename the copy to `Crane1A_Seq`.
2. Double-click `Crane1A_Seq` to open its SCL code.
3. Press **Ctrl+H** (Search and replace) in the code editor.
4. **Find** `Line4Data` → **Replace with** `Crane1AData` → **Replace all**. Confirm every hit changed.
5. Click **Compile** (right-click the block → Compile). Fix nothing — it should be 0 errors.

Crane 1B — repeat: 6. Copy `CraneSequencer` again → Paste → rename `Crane1B_Seq`. 7. Open it, **Ctrl+H**, replace `Line4Data` → `Crane1BData`, **Replace all**, Compile.

Step 6 — Create the IOMap function (the real wiring — the custom part)

1. Project tree → **Program blocks** → double-click **Add new block**.
2. In the dialog choose **Function (FC)**. **Name** = `IOMap`. **Language** = **SCL**. Click **OK**.
3. The block opens. Delete any template lines in the code body.
4. Open `D:\MyGitHub\crane_automation\plc\PLC_1_line1\IOMap.scl` in a text editor, **select all, copy**, and **paste** the whole thing into the `IOMap` code body.
5. Click **Compile**. Any red underline = a `%I`/`%Q` address or DB tag it can't find; fix the module addresses (Step 3) or the DB name until it's 0 errors.

Two wiring gaps in PLC_1 (for real hardware — on the sim you just drive these bits from a watch table, so you can ignore them while simulating):

- **Crane 1B "Loading" (`ls_at[1]`)** — 1B is downstream and has no loading/transfer switch listed in its panel. Set the switch it stops at to collect from `TRVL-1A` and fill that `%I` in `IOMap.scl`.
- **Parking addresses** — the drawings cross-reference them (1A's panel lists `PRK-1B`, 1B's lists `PRK-1A`); double-check each crane's own home sensor.

Step 7 — Wire OB1 (call the three blocks in order)

1. Project tree → **Program blocks** → double-click **Main [OB1]**. It opens in **LAD**.
2. **Network 1:** drag `IOMap` from the tree onto the rung. (No instance DB — it's an FC.)
3. **Network 2:** drag `Crane1A_Seq` onto a new rung. A **Call options** dialog pops up asking for an instance data block → keep the suggested name `Crane1A_Seq_DB` → **OK**.
4. **Network 3:** drag `Crane1B_Seq` on → accept instance DB `Crane1B_Seq_DB` → **OK**.
5. Order must be **IOMap** → **Crane1A_Seq** → **Crane1B_Seq**. Both sequencers run every scan, so the two cranes operate in parallel.

> If OB1 already had a call to the old Line-4 sequencer (from CraneSim), delete that rung.

Step 8 — Remove the Line-4 leftovers (so this isn't "a second Line 4")

1. In **Program blocks**, right-click `Line4Data` → **Delete** (you copied it in Step 4; the originals aren't needed here). Do the same for `PlcData` if present and unused.
2. Right-click the generic `CraneSequencer` → **Delete** (you made per-crane copies in Step 5).
3. Right-click the PLC → **Compile** → **Software (rebuild all blocks)**. Fix any "unknown block" reference that pops from the deletions.

> Keep `typeTask` and `TravelTo` — those are the shared blocks both cranes use.

Step 9 — Compile, download, run

1. **PLCSIM sim only:** right-click the project node → **Properties** → **Protection** → tick **"Support simulation during block compilation"**.
2. Right-click the PLC → **Compile** → **Software (rebuild all blocks)** → expect **0 errors**.
3. Right-click the PLC → **Download to device**. In the Load preview, set **"Stop modules" = Stop all** and, on a fresh DB, **"Delete and replace system data in target"**. Click **Load**, then **Finish**.
4. Set the CPU to **RUN** (Start module in the load dialog, or the RUN button).

Step 10 — Test on the simulator (no real crane needed)

For each crane (`Crane1AData`, then `Crane1BData`), using a **watch table** (right-click *Watch and force tables* → *Add new watch table*):

1. Add `Crane1AData.tasks[1].station`, `.dwell_s`, `task_count`, `cmd_send`, `sts_state`, `ls_at[0]`, `ls_at[2]`, `ls_dip`, `ls_up`, `btn_start`.
2. Set (Modify) `tasks[1].station = 2`, `dwell_s = 6`, `task_count = 1`, `ls_at[0] = TRUE`, `ls_up = TRUE`. `sts_state` should read `0`.
3. Modify `cmd_send → TRUE`. Now walk the switches to advance `sts_state`: make `ls_at[1]` then `ls_dip`, hold `btn_start` 3 s, `ls_up`, `ls_at[2]`, `ls_dip`, wait the dwell, `ls_up`, then `ls_at[0]` — you should step `10 → 20 → … → 90`.
4. Modify `cmd_send → FALSE` → `sts_state` returns to `0`. Repeat on `Crane1BData`. Both reaching 90 independently from one OB1 = parallel operation confirmed.

Step 11 — Real PLC only: security, cert, clock

1. IP is already `192.168.0.111`.
2. A real S7-1500 exposes **only secured** OPC UA endpoints. On the PLC_1 connection in the app set `securityMode`/`securityPolicy` (e.g. `Basic256Sha256` / SignAndEncrypt) to match.
3. Move our client cert (`pki/own/certs/client_certificate.pem`) from the PLC's **rejected** to **trusted** store.
4. **Fix the PLC clock** (OPC UA checks cert validity against the server clock).

See the root `CLAUDE.md` gotchas + `docs/tia_opcua/` for the secured-endpoint + ECONNREFUSED runbooks.

Reference — Crane 1A stations (fully wired)

`station = tank.number + 1`. 0 Park = `I2.6` (PRK-1B), 1 Load = `I1.2` (LOAD-1A), then:

St	Tank	Input	St	Tank	Input
2	`2-1A`	I1.6	7	`2-4A`	I2.3
3	`2-1B`	I1.7	8	`2-4B`	I2.4
4	`2-2`	I2.0	9	`2-5`	I2.5
5	`INS-1A`	I2.1	10	`14-3`	I2.7

St	Tank	Input	St	Tank	Input
6	2-3	I2.2	11	TRVL-1A	I3.0

Hoist I0.3/0.4/0.5; motors R-Up/Down/East/West = Q0.4/0.5/0.6/0.7; R-Hoist/Trolley Q1.0/1.1. **Travel direction: West = forward = right (R-West ← mv_fwd); East = reverse = left (R-East ← mv_rev).**

Reference — Crane 1B stations (Loading TODO)

0 Park = I5.2 (PRK-1A); 12 7-1 = I5.3 · 13 7-2 = I5.4 · 14 14-2 = I5.5 · 15 14-1 = I5.6. Hoist I4.3/4.4/4.5; motors Q1.6/1.7/2.0/2.1; R-Hoist/Trolley Q2.2/2.3.

Reference — OPC UA NodeIds this PLC exposes (for PLC Manager)

Namespace **3**. Bind PLC_1 → line1 cranes 1A/1B in PLC Manager, endpoint opc.tcp://192.168.0.111:4840. Per crane the app addresses its own DB (array bracket **outside** the quotes, **0-based**):

```
ns=3;s="Crane1AData"."tasks"[0]."station"   Int   write   (OPC UA [0] = S7
tasks[1])
ns=3;s="Crane1AData"."tasks"[0]."dwell_s"   Int   write
ns=3;s="Crane1AData"."task_count"           Int   write
ns=3;s="Crane1AData"."cmd_send"             Bool  write
ns=3;s="Crane1AData"."sts_busy"             Bool  read
ns=3;s="Crane1AData"."sts_done"             Bool  read
ns=3;s="Crane1AData"."sts_step"             Int   read
```

Crane 1B is identical with Crane1BData.

Done when…

- [ ] Project saved as PLC_1_Line1; IP 192.168.0.111; OPC UA server activated; DI/DO modules added.
- [ ] Crane1AData + Crane1BData exist, Optimized access ON, *Writable* set on app-written rows.
- [ ] Crane1A_Seq + Crane1B_Seq compile; the only edit was the Line4Data → Crane1xData replace.
- [ ] IOMap pasted + compiles; 1B-Loading / PRK TODOs filled or noted as sim-driven.
- [ ] OB1 calls IOMap → Crane1A_Seq → Crane1B_Seq; Line4Data/PlcData/CraneSequencer deleted.
- [ ] Rebuild-all = 0 errors; CPU in RUN; sim reaches state 90 and back to 0 on **both** cranes.

Alternative: fresh project (instead of duplicating CraneSim)

If you prefer a blank start: **Project** → **New**, add CPU 6ES7511-1AK02, then in Step 4/5 drag typeTask, TravelTo, CraneSequencer from the **CraneAutomation_Lib** global library (make it once from CraneSim — see ../PLC_4_line4/BUILD-STEPS.md), and build Crane1AData/Crane1BData by hand from ../_shared/blocks.md §2. Everything else (IOMap, OB1, test) is the same. This avoids the Step 8 cleanup but makes you type the data block. Deep reference: docs/line4/instructions.md.